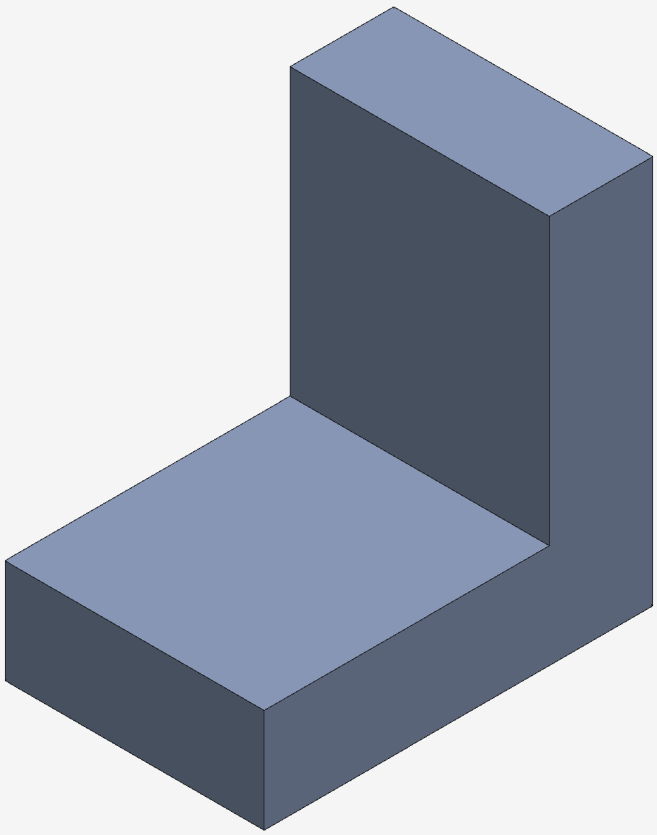


USER MANUAL

Saulitwerx

Open-source parametric 3D CAD

Describes	`main` at commit `a831240`
Built	2026-10-22
Status	Pre-alpha: no desktop app yet

How to use this manual

This manual teaches you to use Saulitwerx, an open-source program for designing mechanical parts on a computer. You don't need to know anything about computer-aided design (CAD) to read it. Where a word has a special meaning, the manual explains it the first time it appears, and again in the glossary at the back.

Saulitwerx is young

Saulitwerx is being built in the open, one small step each night. Today it is **pre-alpha**: the parts that do the real geometry work exist and are tested, but there is no desktop app with windows and buttons yet. That changes what this edition of the manual can offer:

- The **Concepts** part is for everyone. It explains how parametric CAD works, with pictures, so you will be ready when the app arrives.
- The **Tasks** part shows what you can already do, by running real commands and small programs. These sections carry a purple **For developers** band, because they need a terminal and some comfort with installing software.
- Nothing in this manual describes a feature as working unless it works on the version named on the cover. Plans are kept apart, in **Coming next** at the back.

How the book is organized

Part	Read it when you want to
I. Getting started	Know what Saulitwerx is, install it, and make your first picture of a part.
II. Concepts	Understand sketches, constraints, dimensions, parameters, features and solids.
III. Tasks	Do one specific thing. Each section is one task, with numbered steps.
IV. Reference	Look up an exact option, default or allowed value.
V. Help	Fix a problem, or understand an error message.
Back matter	Look up a word, see what changed, or find a topic in the index.

Conventions

You see	It means
`swx-qa render`	Text you type exactly, or text the program prints.
Enter	A key on your keyboard.
sketch	A term defined at this spot. It is also in the glossary and the index.
1	A numbered step. Steps start with what to do, then say what you should see.
RESULT	What you have when a task is done.

> **TIP**
>
> Callouts like this one hold shortcuts and good habits.

> **CAUTION**
>
> Cautions warn about a mistake that is easy to make, and how to avoid it.

> **FOR DEVELOPERS**
>
> Sections with this band run programs from a terminal. You can skip them and still follow every concept.

Which version this describes

The cover names the commit of the `main` branch this edition describes, and the date it was built. The latest edition is always at github.com/glitchwizard/saulitwerx, in `docs/manual/`. **What's new**, near the back, lists what each edition added.

Contents

1 Welcome to Saulitwerx 2
- 1.1 What Saulitwerx is 2
- 1.2 What it can do today 2
- 1.3 Who should use it now 3

2 Installing Saulitwerx 4
- 2.1 What gets installed 4
- 2.2 Install the tools 4
- 2.3 Get the source and build it 5
- 2.4 Other systems 6

3 Your first part 7
- 3.1 Draw a sample part 7
- 3.2 Build a plate with a hole 8
- 3.3 What you just did 9

4 How parametric CAD works 11
- 4.1 Solids 11
- 4.2 Units 11
- 4.3 Sketches 12
- 4.4 Constraints 12
- 4.5 Dimensions 12
- 4.6 Parameters 13
- 4.7 Features 13
- 4.8 History 13
- 4.9 Putting it together 14

5 Sketching 16
- 5.1 Solve a sketch 16
- 5.2 How the sample sketch is built 17
- 5.3 Change a dimension 17
- 5.4 Common mistakes 18

6 Solids and STEP files 19
- 6.1 Make a solid and save it as STEP 19
- 6.2 Combine solids 20
- 6.3 Open a STEP file from another program 20

7 Pictures of parts 21
- 7.1 Draw a STEP file 21
- 7.2 Choose the size 21
- 7.3 Choose which edges are outlined 21
- 7.4 Draw a built-in sample 22

8 Parameters and history 23
- 8.1 See what a change rebuilds 23
- 8.2 How the sample is built 23
- 8.3 Common mistakes 24

9 Reference 26
9.1 `swx-qa render` 26
9.2 Build options 26
9.3 Geometry kernel operations 27
9.4 Sketch entities and constraints 27
9.5 Solver settings and report 28
9.6 File formats and units 28
9.7 Keyboard and mouse 28
10 Help 30
10.1 Troubleshooting 30
10.2 Error messages 30
10.3 Questions 31
10.4 Known limitations 32
Glossary 34
What's new 35
2026-10-22: first edition 35
Coming next 36
Credits and licenses 38
Index 39

PART I

Getting started

What Saulitwerx is, what it can do today, how to install it, and a first session that ends with a picture of a real part.

CHAPTER 1

Welcome to Saulitwerx

1.1 What Saulitwerx is

Saulitwerx is a program for designing solid mechanical parts on a computer: brackets, plates, shafts, housings, and anything else you might machine, cast or 3D-print. Programs like this are called *CAD* (computer-aided design). Saulitwerx is free and open source, so anyone can use it, study it, change it, and build it into other software.

Saulitwerx is a *parametric* modeler. You don't sculpt a shape by pushing points around. Instead you describe how to build it, step by step, with measurements: "draw a 40 by 25 mm rectangle, push it up 5 mm, cut a 10 mm hole through the middle." Saulitwerx remembers those steps and their numbers. Change the hole to 12 mm later, and the part rebuilds itself to match. Chapter 4 explains how that works.

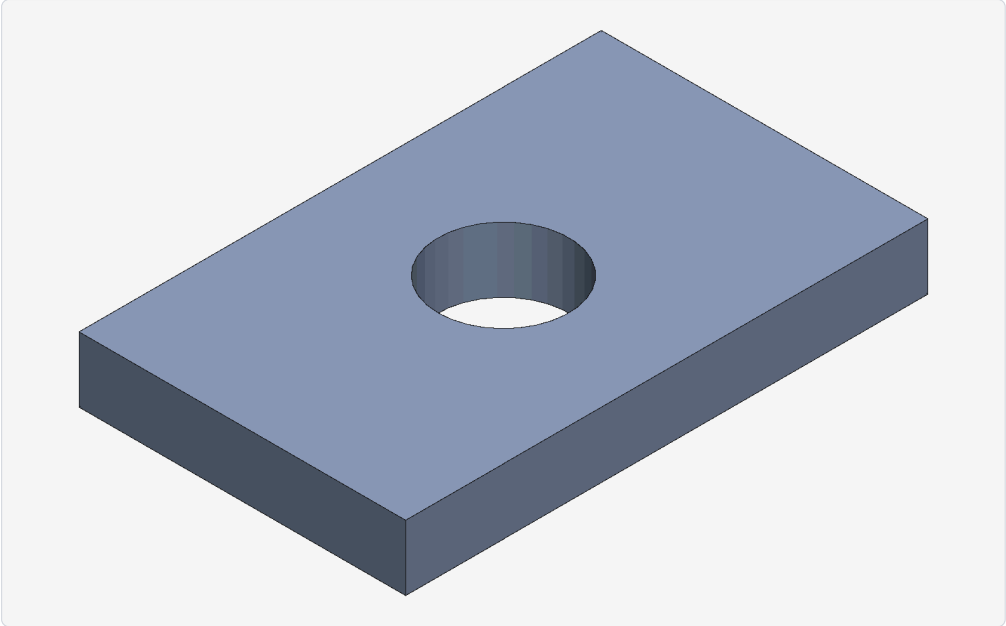


Figure 1.1 A plate with a hole, made by Saulitwerx's geometry engine and drawn by its renderer. You will make this part yourself in Chapter 3.

1.2 What it can do today

Saulitwerx is built in small, tested steps, in the order of a public plan (the *roadmap*). Here is where it stands for this edition.

Area	What exists	Use it
Solids	Make boxes, cylinders and spheres; combine them (add, subtract, keep the overlap); measure volume, surface area and size.	NOW

Area	What exists	Use it
Exchange	Save a solid as a STEP file, and read STEP files back.	NOW
Pictures	Draw a solid or a STEP file to a PNG picture with `swx-qa render`.	NOW
Sketching	Lines, circles, arcs and points, with the constraints coincident, horizontal, vertical, fixed, distance, radius and diameter, and a solver that moves the shape to satisfy them.	NOW
History	Keep features in order with named parameters, and work out which features a change affects.	NOW
Desktop app	Windows, menus, a 3D view and mouse control.	LATER
Part files	Saving your design with its full history in a Saulitwerx part file.	LATER
Features	Extrude, revolve, fillet, chamfer, hole, shell and more, driven by sketches.	LATER
Assemblies, drawings	Putting parts together, and making 2D shop drawings.	LATER

Everything marked NOW is a building block that programmers can call today. The **Tasks** part shows each one with a real command or a short program you can run. Everything marked LATER is described in **Coming next** at the back, and nowhere else.

1.3 Who should use it now

- **If you are new to CAD,** read Parts I and II. You will learn the ideas every parametric modeler is built on, and you will be ready the day the desktop app arrives.
- **If you are comfortable in a terminal,** install Saulitwerx (Chapter 2) and follow Chapter 3. It takes about 15 minutes and ends with a picture of a part you built.
- **If you write software,** the **For developers** sections show how to call each building block from Rust.

> **NOTE**
>
> Saulitwerx reads and writes only open, documented file formats. It will never open another program's own native files. To bring in a design from elsewhere, export it as STEP first.

CHAPTER 2

Installing Saulitwerx

There is no installer yet. Until the first release, you build Saulitwerx from its source code. That sounds harder than it is: three tools to install, then one command builds everything. Plan on about 10 minutes, most of it waiting for downloads.

YOU WILL	Install the tools Saulitwerx needs, download its source code, build it, and check that it works.
YOU NEED	A computer running Linux (these steps were tested on Ubuntu 24.04), an internet connection, and permission to install software (`sudo`).
TIME	About 10 minutes.

2.1 What gets installed

Tool	Why Saulitwerx needs it	Required?
Git	Downloads the source code and keeps it up to date.	Yes
Rust (1.80 or newer)	The programming language Saulitwerx is written in. Its tool `cargo` builds and runs the program.	Yes
C++ compiler	Builds a small bridge between Saulitwerx and the geometry engine.	For solids and STEP
OpenCascade (OCCT) 7.6 or newer	The open-source geometry engine that does the solid modeling.	For solids and STEP

Without OpenCascade you can still sketch, try the document model, and draw the built-in sample parts. To make your own solids or read and write STEP files, you need it.

2.2 Install the tools

1. Open a terminal. On Ubuntu, press `Ctrl` + `Alt` + `T`. A window with a command prompt appears.
2. Install Git, a C++ compiler and OpenCascade:

   TERMINAL

   ```
   sudo apt-get update
   sudo apt-get install -y git build-essential curl \
      libocct-foundation-dev libocct-modeling-data-dev \
      libocct-modeling-algorithms-dev libocct-visualization-dev \
      libocct-data-exchange-dev
   ```

   Type your password when asked. The terminal prints a list of packages and ends at the prompt again.
3. Install Rust with its official installer, and accept the default choices when it asks:

   TERMINAL

   ```
   curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
   ```

It ends with `Rust is installed now. Great!`

4. Close the terminal and open a new one, so it finds the new tools. Then check them:

```
TERMINAL
cargo --version
```

You should see a version line such as `cargo 1.97.0 (…)`. Any version 1.80 or newer works.

2.3 Get the source and build it

1. Download the source code into a folder named `saulitwerx`, and go into it:

```
TERMINAL
git clone https://github.com/glitchwizard/saulitwerx
cd saulitwerx
```

2. Build Saulitwerx and run its own checks:

```
TERMINAL
cargo test --workspace
```

The first build downloads a few small libraries and takes a minute or two. It ends with several lines that start `test result: ok.` and none that say `FAILED`.

3. Build the picture tool with OpenCascade support:

```
TERMINAL
cargo build --release -p swx-qa --features occt
```

It ends with a line that starts `Finished`. The tool is now at `target/release/swx-qa`.

4. Check that the picture tool runs:

```
TERMINAL
target/release/swx-qa --help
```

You should see its help text, which starts `Usage: swx-qa render`.

RESULT Saulitwerx is built, its tests pass, and the `swx-qa` picture tool is ready. Go on to Chapter 3.

> **TIP** **Staying up to date**
>
> Saulitwerx changes most nights. To get the newest version, go into the `saulitwerx` folder and run `git pull`, then build again.

> **CAUTION** **Commands run from the `saulitwerx` folder**
>
> Every command in this manual assumes your terminal is in the `saulitwerx` folder. If a command says it can't find a file, run `cd saulitwerx` first. Chapter 10 lists other problems and their fixes.

2.4 Other systems

The build is regularly checked only on Ubuntu Linux. Saulitwerx aims to run on macOS and Windows too, and the same steps should work there once Rust and OpenCascade are installed, but they are untested. Without OpenCascade, leave out `--features occt` in step 3.

CHAPTER 3

Your first part

In this tutorial you draw a sample part, then build a real one, a plate with a hole through it, measure it, save it in a standard file format, and draw it. Along the way you will meet the ideas the rest of the manual builds on.

YOU WILL	Draw a built-in sample part, build a plate with a hole, check its measurements, save it as a STEP file, and draw it.
YOU NEED	Saulitwerx built with OpenCascade support (Chapter 2), and a terminal in the `saulitwerx` folder.
TIME	About 15 minutes.

3.1 Draw a sample part

Saulitwerx comes with three sample parts built in: a cube, a cylinder and an L-shaped bracket. They need nothing else, so they are a quick way to check that drawing works.

1. Draw the cube to a picture file named `cube.png`:

   TERMINAL

   ```
   target/release/swx-qa render --fixture cube -o cube.png
   ```

   The tool answers with one line:

   OUTPUT

   ```
   wrote cube.png (800x600, 27669 bytes)
   ```

2. Open `cube.png` in any image viewer. On Ubuntu, type `xdg-open cube.png`. You should see a grey-blue cube on a light grey background, like Figure 3.1.

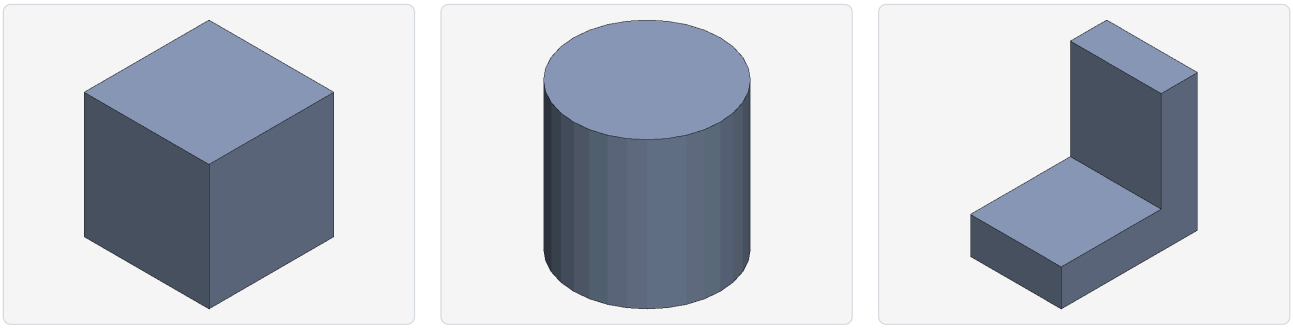


Figure 3.1 The three built-in sample parts: `cube`, `cylinder` and `l-bracket`.

The picture is always drawn from the same corner, looking down at an angle, so that three sides of a box show at once. Each flat face gets one even shade, and sharp edges get a dark outline. This kind of view is called *isometric*.

> **TIP**
>
> Try `--fixture cylinder` and `--fixture l-bracket` too. Add `--size 1600x1200` for a bigger, sharper picture.

3.2 Build a plate with a hole

Now build a real part. Saulitwerx doesn't have a window to draw in yet, so this part comes from a small program that ships with Saulitwerx. The program asks the geometry engine for three things:

1. a *solid* block 40 mm long, 25 mm wide and 5 mm thick;
2. a cylinder 10 mm across, standing where the hole should go, a little taller than the plate;
3. the block **minus** the cylinder. Taking one solid away from another is called a *cut*.

1. Run the program, and tell it to save the part as `plate.step`:

   TERMINAL

   ```
   cargo run -p swx-kernel --features occt --example plate_with_hole -- plate.step
   ```

   After a few seconds of building, it prints the part's measurements. The geometry engine also prints a short report about writing the file, framed in rows of `*`; you can ignore it.

   OUTPUT

   ```
   volume        4607.30 mm³
   surface area  2650.00 mm²
   faces         7
   bounding box  [0.0, 0.0, 0.0] to [40.0, 25.0, 5.0]
   …  (the geometry engine's file report)
   wrote plate.step (read back: 4607.30 mm³, 7 faces)
   ```

2. Check the numbers. The block holds $40 \times 25 \times 5 = 5000$ mm³. The hole removes $\pi \times 5^2 \times 5 \approx 392.7$ mm³. $5000 - 392.7 = 4607.3$, which is what the program reports. The part has 7 faces: top, bottom, four sides and the inside of the hole.

3. Draw the part:

   TERMINAL

   ```
   target/release/swx-qa render --step plate.step --size 1600x1000 -o plate.png
   ```

   The tool answers `wrote plate.png (1600x1000, …)`.

4. Open `plate.png`. You should see the plate of Figure 3.2, with the hole showing light background through it.

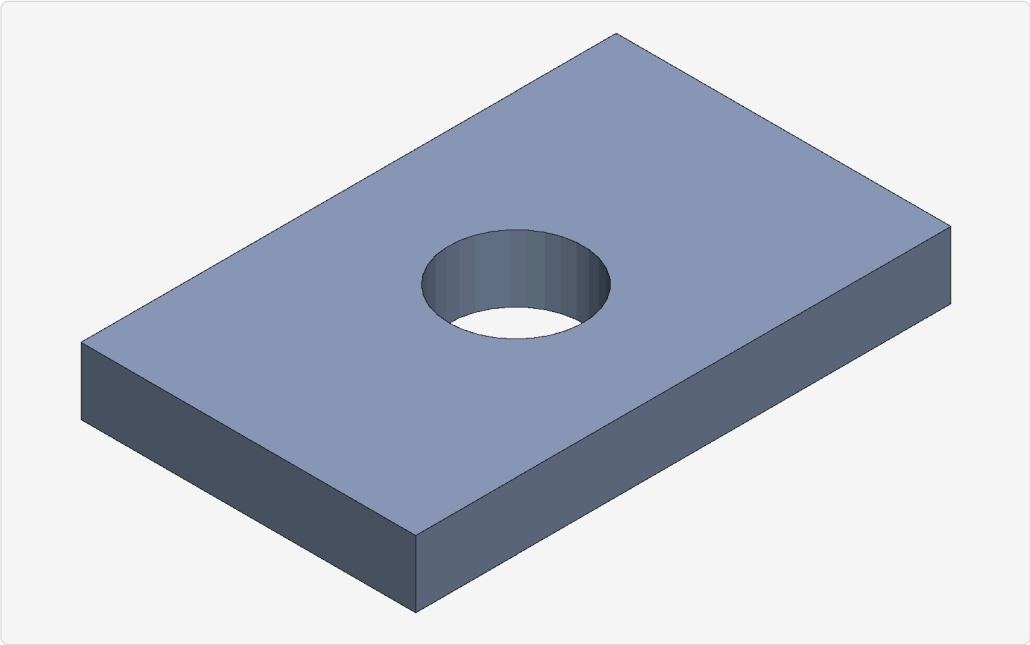


Figure 3.2 The plate you built, drawn from the STEP file the program saved.

RESULT You built a solid part, checked its volume, saved it as `plate.step`, and drew it as `plate.png`.

3.3 What you just did

The file `plate.step` is in the *STEP* format, an international standard (ISO 10303) for exchanging 3D parts. Almost every CAD program can open it, so you can take this plate into another tool, send it to a machine shop, or prepare it for 3D printing.

What you did **not** see yet is the parametric part of Saulitwerx: the numbers 40, 25, 5 and 10 are typed into the program, not stored as named, editable parameters. That comes with the part features and part files on the roadmap. Chapter 4 explains the ideas behind it, and Chapter 8 shows the piece of it that already works.

> **FOR DEVELOPERS**
>
> The program you ran is `crates/swx-kernel/examples/plate_with_hole.rs`. Change `5.0` (the hole radius) in the `make_cylinder` line to `8.0`, run step 1 again, and the volume drops to 3994.69 mm³. Chapter 6 explains each call.

PART II

Concepts

The ideas behind parametric CAD, in plain words. Read this before the task chapters; every procedure later relies on these terms.

CHAPTER 4

How parametric CAD works

Every parametric modeler, Saulitwerx included, is built on a handful of ideas. Once you know them, the commands make sense on their own. This chapter explains each idea in the order you meet it when you design a part. For each one it says what it is, why it matters, and where it stands in Saulitwerx today.

4.1 Solids

A *solid* is a closed 3D shape with an inside and an outside, like a real object. Because Saulitwerx knows what is inside, it can tell you how much material a part holds (its *volume*), how much surface it has to paint (its *surface area*), and how big a box it would fit in (its *bounding box*).

The outside of a solid is made of *faces*: flat or curved patches of surface. Where two faces meet there is an *edge*. The plate from Chapter 3 has 7 faces: top, bottom, four sides, and the round inside of the hole.

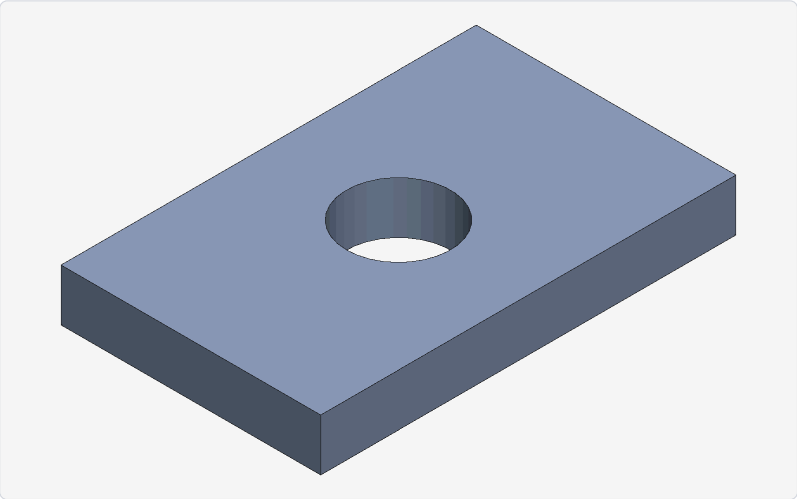


Figure 4.1 A solid. Its faces are shaded; its edges are outlined. Volume 4607.30 mm³, 7 faces.

In Saulitwerx today: the geometry engine makes boxes, cylinders and spheres, and measures volume, surface area, bounding box and face count. Chapter 6 shows how.

4.2 Units

Every length in a part has a *unit*. In Saulitwerx the default is the millimeter (mm), and all of today's building blocks work in millimeters: a box 40 long is 40 mm long, and volumes come out in mm³. Angles are in degrees where you type them, and in radians inside the sketch geometry.

In Saulitwerx today: everything is in millimeters. STEP files are written in millimeters too. Choosing units per part comes with part files.

4.3 Sketches

Most solids start as a flat drawing. A *sketch* is a 2D drawing on a flat plane, made of simple pieces: points, straight lines, circles and arcs. Together these are called sketch *geometry* or *entities*.

A sketch for the plate would be a rectangle (four lines) and a circle. You don't need to draw it accurately. A rough shape is fine, because the next two ideas, constraints and dimensions, make it exact.

In Saulitwerx today: sketches with points, lines, circles and arcs exist as a building block (Chapter 5). There is no drawing window yet, so sketches are written as a short program.

> **NOTE Pictures of sketches**
>
> Sketch pictures need a sketch-drawing tool that is planned but not built yet. Until it arrives, this edition shows sketches as their solved coordinates (Chapter 5).

4.4 Constraints

A *constraint* is a rule about the shape of a sketch: "this line is horizontal," "these two corners touch," "this point stays at the origin." Constraints carry the **intent** of a design. If you later change a size, a line that must stay horizontal stays horizontal.

Constraint	Rule it keeps
Coincident	Two points are at the same place. Use it to join the end of one line to the start of the next.
Horizontal	A line runs left–right; or two points are at the same height.
Vertical	A line runs up–down; or two points are directly above each other.
Fixed	A point stays at exact coordinates. Use it to anchor a sketch.

Saulitwerx satisfies constraints with a *solver*: a calculation that nudges every point until all the rules hold at once. You give it a rough drawing and the rules; it gives back the exact shape.

4.5 Dimensions

A *dimension* is a constraint with a number: a length, a distance, a radius or a diameter. "The bottom line is 40 mm long." "The hole is 10 mm across." Dimensions fix the size of a sketch, the way the plain constraints above fix its shape.

Dimension	Number it sets
Distance	The straight-line distance between two points, for example the two ends of a line.
Radius	The radius of a circle or arc.
Diameter	The diameter of a circle or arc: twice the radius.

When a sketch has enough constraints and dimensions that nothing can move any more, it is *fully constrained*. Each way a piece can still move is called a *degree of freedom*. A free point on a sheet has two (left–right and up–down); a free circle has three (its center's two, and its size).

> **TIP**
>
> Constrain the shape first (coincident, horizontal, vertical), then add dimensions. Then anchor one point with fixed, so the whole sketch can't slide around.

In Saulitwerx today: all of the constraints and dimensions in these two tables work, and the solver reports whether it found a shape that meets them. Reporting how many degrees of freedom are left, and which rules conflict, is the next item on the roadmap.

4.6 Parameters

A *parameter* is a named number, like `thickness = 5` or `hole_dia = 10`. Features and dimensions refer to parameters by name instead of holding a bare number. Change the parameter, and everything that uses it follows.

This is the "parametric" in parametric CAD. One plate design, with its thickness as a parameter, is really a whole family of plates: set `thickness` to 3, 5 or 8 and rebuild.

In Saulitwerx today: parts hold named parameters, and features refer to them by name (Chapter 8). Expressions such as `d3 = d1 / 2 + 5 mm`, and automatic parameters for every dimension, are on the roadmap.

4.7 Features

A *feature* is one step in building a part: a sketch, or something done to the solid, such as pushing a sketch out into 3D or cutting a hole. Each feature records **what** it does and **which** sketch and parameters it uses, not just the shape that resulted.

Figure 4.2 shows the idea of a cut, done directly with the geometry engine: a block, minus a cylinder, gives the plate. A future **extrude** feature will make the block from a rectangle sketch, and a **cut** feature will make the hole from the circle.

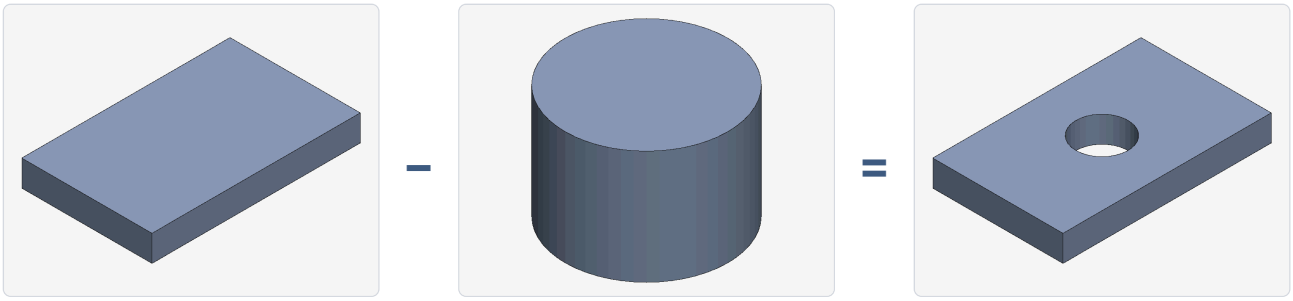


Figure 4.2 A cut takes one solid away from another. Each picture is scaled to fill its frame, so the cylinder looks larger than it is.

There are three ways to combine two solids. Together they are called *Boolean operations*:

- **Union** (join): keep everything that is in either solid.
- **Cut** (subtract): keep the first solid, minus anything the second one overlaps.
- **Intersect**: keep only the part that is in both.

4.8 History

A part's *history*, also called its *feature tree*, is its list of features in the order they were made. Each feature can use only what came before it: a cut needs a solid to cut into, and an extrude needs a sketch to push.

#	Feature	Uses
0	Sketch1	nothing: a rectangle on a plane
1	Extrude1	Sketch1, and the parameter `thickness`
2	Sketch2	nothing: a profile for a pin
3	Revolve1	Sketch2, and the parameter `sweep`

Because the history remembers how each feature was made, Saulitwerx can **rebuild** a part after a change. It doesn't rebuild everything. It works out which features depend on what changed, and redoes only those. Change `thickness`, and only Extrude1 rebuilds; edit Sketch2, and Sketch2 and Revolve1 rebuild.

In Saulitwerx today: the history and the "what needs rebuilding" logic exist (Chapter 8), using the exact example in the table above. Running the rebuild on real geometry comes with the part features.

4.9 Putting it together

Here is how the ideas line up when you design a part, once the app exists:

1. Make a **sketch** on a plane, roughly the shape you want.
2. Add **constraints** to fix its shape, and **dimensions** to fix its size, until it is fully constrained.
3. Turn the sketch into a **solid** with a **feature**, such as extrude.
4. Add more features (cuts, holes, rounded edges) to finish the part. They form its **history**.
5. Change a **parameter** or a dimension later, and the part rebuilds to match.

PART III

Tasks

How to do things, one task per section. Until the desktop app arrives, every task here runs a real command or a small program, and is marked for developers.

CHAPTER 5

Sketching

> **FOR DEVELOPERS**
>
> Until the desktop app's sketch mode arrives, sketches are written as a short Rust program against the `swx-sketch` library. This chapter runs one that ships with Saulitwerx.

5.1 Solve a sketch

The sketch for the plate of Chapter 3 is a 40 × 25 mm rectangle with a 10 mm circle in the middle. In this task you run a program that draws that sketch **roughly**, adds the constraints and dimensions, and asks the solver for the exact shape.

YOU WILL	Run the sample sketch, read the solver's report, and see the rough shape become exact.
YOU NEED	Saulitwerx built (Chapter 2). OpenCascade is not needed.
TIME	5 minutes.

1. Run the sample sketch:

   TERMINAL
   ```
   cargo run -p swx-sketch --example plate_sketch
   ```

   It prints the solver's report, then each line and the circle:

   OUTPUT
   ```
   solved: true (Converged, 3 iterations, residual 5.3e-14)
   bottom (  0.00,   0.00) -> ( 40.00,   0.00)  length 40.00 mm
   right  ( 40.00,   0.00) -> ( 40.00,  25.00)  length 25.00 mm
   top    ( 40.00,  25.00) -> (  0.00,  25.00)  length 40.00 mm
   left   (  0.00,  25.00) -> (  0.00,   0.00)  length 25.00 mm
   hole   center (20.00, 12.50)  diameter 10.00 mm
   ```

2. Read the first line. `solved: true` and `Converged` mean the solver found a shape that meets every rule. `residual` is how far off it still is, here about 0.00000000000005 mm: exact for any practical purpose.
3. Compare the result with the rough drawing in the program (below). The bottom line was drawn 38 mm long and tilted; it is now exactly 40 mm and level.

RESULT A fully sized, square-cornered sketch: a 40 × 25 mm rectangle with its lower-left corner at the origin, and a 10 mm circle centered at (20, 12.5).

5.2 How the sample sketch is built

The program is `crates/swx-sketch/examples/plate_sketch.rs`. It works in four stages, the same ones you will use in the app. The code below is shortened from the file.

1. Draw roughly. Four lines and a circle, deliberately off-size:

```
let bottom = s.add(Geometry::Line(Line::new(p(0.0, 0.0)?, p(38.0, 1.0)?)?))?;
let right = s.add(Geometry::Line(Line::new(p(38.0, 1.0)?, p(39.0, 22.0)?)?))?;
let top = s.add(Geometry::Line(Line::new(p(39.0, 22.0)?, p(1.0, 24.0)?)?))?;
let left = s.add(Geometry::Line(Line::new(p(1.0, 24.0)?, p(0.0, 0.0)?)?))?;
let hole = s.add(Geometry::Circle(Circle::new(p(18.0, 11.0)?, 4.0)?))?;
```

2. Fix the shape. Join each line's end to the next line's start with **coincident**, then make two lines horizontal and two vertical:

```
for (a, b) in [(bottom, right), (right, top), (top, left), (left, bottom)] {
    s.add_constraint(Constraint::Coincident(PointRef::End(a), PointRef::Start(b)))?;
}
s.add_constraint(Constraint::Horizontal(bottom))?;
s.add_constraint(Constraint::Vertical(left))?;   // and likewise top, right
```

3. Anchor and size it. Pin the lower-left corner to the origin with **fixed**, and set two **distances** and the hole's **diameter**:

```
s.add_constraint(Constraint::Fixed { point: PointRef::Start(bottom), x: 0.0, y: 0.0 })?;
s.add_constraint(Constraint::Distance {
    a: PointRef::Start(bottom), b: PointRef::End(bottom), value: 40.0 })?;
s.add_constraint(Constraint::Diameter { geom: hole, value: 10.0 })?;
```

4. Solve. One call moves everything into place and returns a report:

```
let report = s.solve(SolveOptions::default())?;
```

A `PointRef` names a point the constraint acts on: `Start(line)`, `End(line)`, `Center(circle)` or `Point(point)`. Section 9.4 lists every entity and constraint with its exact form.

5.3 Change a dimension

1. Open `crates/swx-sketch/examples/plate_sketch.rs` in a text editor.
2. Find `value: 40.0` and change it to `value: 60.0`. Save the file.
3. Run the sample again (step 1 above). The bottom and top lines now report `length 60.00 mm`; the right line moves to x = 60. The hole stays at (20, 12.5), because it is fixed there, not centered.
4. Change the value back to `40.0` when you are done.

> **TIP**
>
> To keep the hole centered whatever the width, the sketch would need a **midpoint** or **symmetric** constraint. Those are next on the roadmap.

5.4 Common mistakes

- **A value of zero or less.** Distance, radius and diameter must be greater than zero. Adding `value: 0.0` fails with `value must be > 0, got 0`.
- **The wrong kind of point.** `PointRef::Start` and `End` work only on lines, and `Center` only on circles and arcs. Using `Center` on a line fails with `geometry 0 has a different kind`.
- **Rules that fight each other.** Two fixed points 30 mm apart with a distance of 40 mm between them can't both hold. The solver then stops with `solved: false` and a reason of `Stagnated`, and moves the points as close to meeting the rules as it can. Saulitwerx can't yet say **which** rules conflict; that is on the roadmap.

CHAPTER 6

Solids and STEP files

> **FOR DEVELOPERS**
>
> This chapter calls the geometry engine, `swx-kernel`, from Rust. It needs OpenCascade and the `occt` build feature (Chapter 2).

The part of a CAD program that does the 3D math is its *geometry kernel*. Saulitwerx talks to its kernel through one interface, the `Kernel` trait, so the rest of the program never depends on which kernel is underneath. Today that kernel is OpenCascade, reached through `OcctKernel`.

6.1 Make a solid and save it as STEP

YOU WILL	Build the plate with a hole, measure it, save it as STEP and read it back.
YOU NEED	Saulitwerx built with OpenCascade (Chapter 2).
TIME	5 minutes.

You already ran this program in Chapter 3. Here is what each part of it does. The file is `crates/swx-kernel/examples/plate_with_hole.rs`.

1. **Make the two starting solids.** A box is given by its lowest corner and its three sizes; a cylinder stands on +Z, given by the center of its base, its radius and its height:

```
let plate = k.make_box([0.0, 0.0, 0.0], [40.0, 25.0, 5.0])?;
let tool  = k.make_cylinder([20.0, 12.5, -1.0], 5.0, 7.0)?;
```

2. **Cut one from the other.** The cylinder starts 1 mm below the plate and ends 1 mm above it, so the hole goes cleanly all the way through:

```
let part = k.cut(&plate, &tool)?;
```

3. **Measure the result** with `volume`, `surface_area`, `face_count` and `bounding_box`. The program prints them; Chapter 3 checks them by hand.

4. **Save and read back.** `export_step` writes the file, in millimeters; `import_step` reads it back as one solid. Matching volume and face count show nothing was lost:

```
k.export_step(&part, &out)?;
let back = k.import_step(&out)?;
```

RESULT A STEP file of the plate, checked by reading it back: `wrote plate.step (read back: 4607.30 mm³, 7 faces)`.

> **TIP Make the tool overlap**
>
> When you cut a hole, make the tool solid a little longer than the part on both sides. A tool that ends exactly on a face can leave a paper-thin skin, or confuse the kernel.

6.2 Combine solids

The kernel has three Boolean operations (Section 4.7). Each takes two solids and returns a new one; the inputs are left unchanged.

Call	Result
`k.union(&a, &b)`	Everything in `a` or `b`.
`k.cut(&a, &b)`	`a` minus whatever `b` overlaps.
`k.intersect(&a, &b)`	Only what is in both.

> **CAUTION**
>
> Sizes must be finite and greater than zero, and positions finite. A box with a zero size is refused before it reaches the kernel, with `invalid input: box dx must be finite and > 0, got 0`.

6.3 Open a STEP file from another program

`import_step` reads any STEP file, and `swx-qa render --step` draws one (Chapter 7). Everything in the file comes in as one shape. Saulitwerx can't edit an imported shape yet; using it as a starting feature is on the roadmap.

CHAPTER 7

Pictures of parts

`swx-qa render` draws a part to a PNG picture file. It needs no graphics card and no window, and the same input always gives the same picture, byte for byte. The project uses it to show its work in reviews; this manual's part pictures come from it too.

7.1 Draw a STEP file

YOU WILL Draw any STEP file to a PNG, at the size you want.
YOU NEED `swx-qa` built with OpenCascade (Chapter 2), and a STEP file. `plate.step` from Chapter 3 works.

1. Run `render` with the STEP file, a size and an output name:

   TERMINAL

   ```
   target/release/swx-qa render --step plate.step --size 1600x1000 -o plate.png
   ```

   It answers `wrote plate.png (1600x1000, …)`, with the file size in bytes.

2. Open `plate.png`. The part fills the picture, seen from the same corner as every other render.

RESULT A PNG of the part, ready to share or to put in a document.

The renderer chooses how finely to break curved faces into flat pieces for you: the mesh stays within 0.1 % of the part's overall size of the true surface.

7.2 Choose the size

`--size` takes the width and height in pixels, written `W` `x` `H`, for example `1600x1000`. The default is `800x600`; the largest is `8192x8192`. For a picture you will print, allow about 80 pixels per centimeter (200 per inch) of printed width: a picture printed 10 cm wide needs about 800 pixels.

7.3 Choose which edges are outlined

The renderer outlines the edges where faces meet at a sharp angle. `--crease` sets how sharp, in degrees, from 0 to 180; the default is 30. Where the surface bends by more than this angle from one face to the next, the edge gets a dark line. The edges of a box bend by 90°, so they are always outlined; the many small facets of a curved face bend by only a few degrees, so they are not. A smaller value outlines more edges; a larger value outlines fewer.

> **TIP**
>
> On a curved part that looks striped with lines, raise `--crease` (try 45). If an edge you want to see has no outline, lower it.

7.4 Draw a built-in sample

`--fixture` draws one of three built-in parts instead of a file: `cube` (10 mm), `cylinder` (5 mm radius, 10 mm tall) or `l-bracket` (30 mm legs, 8 mm thick, 20 mm deep). They need no OpenCascade, so they are a quick way to test the tool.

TERMINAL

```
target/release/swx-qa render --fixture l-bracket -o bracket.png
```

Section 9.1 lists every option.

CHAPTER 8

Parameters and history

> **FOR DEVELOPERS**
>
> This chapter uses the document model, `swx-core`, from Rust. It needs no OpenCascade.

The document model holds what makes a part parametric: its named parameters and its ordered list of features (Section 4.6, Section 4.8). It doesn't make geometry itself. Its job is to keep the history valid and to work out what must be rebuilt after a change.

8.1 See what a change rebuilds

YOU WILL	Build a four-feature history and ask which features a change affects.
YOU NEED	Saulitwerx built (Chapter 2).
TIME	5 minutes.

1. Run the sample:

   TERMINAL

   ```
   cargo run -p swx-core --example feature_history
   ```

   OUTPUT

   ```
   history:
     0  Sketch1
     1  Extrude1
     2  Sketch2
     3  Revolve1
   change thickness -> rebuild Extrude1
   edit Sketch2     -> rebuild Sketch2, Revolve1
   add Extrude2 with depth `height` -> error: unknown parameter `height`
   ```

2. Read the history. It is the table in Section 4.8: two sketches, an extrude that uses `thickness`, and a revolve that uses `sweep`.
3. Read the two rebuild lines. Changing `thickness` touches only Extrude1, the one feature that uses it. Editing Sketch2 rebuilds Sketch2 and Revolve1, which consumes it; Sketch1 and Extrude1 are left alone.
4. Read the last line. A feature may use only parameters that exist, so adding an extrude whose depth is `height` is refused.

RESULT You saw Saulitwerx decide, from the history alone, the smallest set of features a change needs rebuilt.

8.2 How the sample is built

The program is `crates/swx-core/examples/feature_history.rs`. A `Document` holds parameters and features:

```
let mut doc = Document::new();
doc.set_param("thickness", 5.0);
let plate_sketch = doc.add_feature("Sketch1", FeatureKind::Sketch)?;
doc.add_feature("Extrude1", FeatureKind::Extrude {
    profile: plate_sketch,
    depth: "thickness".into(),
})?;
```

Then two questions give the rebuild lists, in history order:

```
doc.dirty_after_param_change("thickness");   // [Extrude1]
doc.dirty_after_feature_edit(pin_sketch)?;    // [Sketch2, Revolve1]
```

The features that need rebuilding are called *dirty*. Today's feature kinds are `Sketch`, `Extrude` (with a depth parameter) and `Revolve` (with an angle parameter, in degrees). They are records in the history only; making their geometry comes with the part features on the roadmap.

8.3 Common mistakes

- **Using a parameter before it exists.** Call `set_param` first; otherwise `add_feature` fails with `unknown parameter`.
- **Using a feature that comes later.** A feature may use only features already in the history. Otherwise `add_feature` fails with `feature N references feature M which is not earlier in the history`.

PART IV

Reference

Every command, option, operation and setting that exists today, with its default and its allowed values.

CHAPTER 9

Reference

9.1 `swx-qa render`

Draws a part as an isometric, flat-shaded PNG with its sharp edges outlined.

FORM

```
swx-qa render (--fixture <name> | --step <file.step>) -o <out.png> [options]
```

Option	What it does	Default
`--fixture <name>`	Draws a built-in part: `cube`, `cylinder` or `l-bracket`. Give this or `--step`, not both.	—
`--step <file>`	Draws a STEP file. Needs a build with `--features occt`.	—
`-o`, `--output <file>`	The PNG to write. Required. An existing file is replaced.	—
`--size <W>x<H>`	Picture size in pixels. Each side from 1 to 8192.	`800x600`
`--crease <deg>`	Outline edges where the surface bends by more than this many degrees. 0 to 180.	`30`
`-h`, `--help`	Prints the help text and does nothing else.	—

On success it prints `wrote <file> (<W>x<H>, <bytes> bytes)` and exits with status 0. On an error it prints `swx-qa:` and the message (Section 10.2) and exits with status 1, without writing a file.

Built-in part	Shape
`cube`	A 10 mm cube with one corner at the origin.
`cylinder`	Radius 5 mm, height 10 mm, standing on +Z; 32 flat segments around.
`l-bracket`	An L profile with 30 mm legs and 8 mm thickness, 20 mm deep.

9.2 Build options

Command	What it builds
`cargo test --workspace`	Everything without OpenCascade, and runs every test.
`cargo build --release -p swx-qa --features occt`	The `swx-qa` tool with STEP input, at `target/release/swx-qa`.
`cargo run -p swx-sketch --example plate_sketch`	Runs the sample sketch (Chapter 5).
`cargo run -p swx-kernel --features occt --example plate_with_hole -- <file.step>`	Builds the plate and saves it (Chapter 6). The file defaults to `plate.step`.
`cargo run -p swx-kernel --features occt --example plate_stages -- <dir>`	Saves the block, the cylinder and the plate as three STEP files in `<dir>`.
`cargo run -p swx-core --example feature_history`	Runs the history sample (Chapter 8).

The `occt` feature switches on OpenCascade. It is off by default, so a plain build needs only Rust.

9.3 Geometry kernel operations

Operation	What it does and what it accepts
`make_box(corner, size)`	A box with its lowest corner at `corner` and sides `size` = [x, y, z]. Sizes > 0.
`make_cylinder(base, r, h)`	A cylinder along +Z with its base circle centered at `base`. Radius and height > 0.
`make_sphere(center, r)`	A sphere. Radius > 0.
`union(a, b)`, `cut(a, b)`, `intersect(a, b)`	Boolean operations (Section 4.7). Inputs unchanged.
`volume(s)`, `surface_area(s)`	In mm³ and mm².
`bounding_box(s)`	The smallest box, aligned with the axes, that holds the solid: its `min` and `max` corners.
`face_count(s)`	The number of faces.
`tessellate(s, deflection)`	Breaks the surface into triangles, no farther than `deflection` (mm, > 0) from the true surface.
`export_step(s, path)`	Writes a STEP file in millimeters, replacing any existing file.
`import_step(path)`	Reads everything in a STEP file as one shape.

All positions must be finite numbers; all sizes finite and greater than zero.

9.4 Sketch entities and constraints

Entity	Defined by
`Point`	x, y.
`Line`	A start point and an end point.
`Circle`	A center point and a radius > 0.
`Arc`	A center, a radius > 0, and start and end angles in radians, running counter-clockwise; the end angle must be after the start.

Constraint	Rule
`Coincident(p, q)`	Points `p` and `q` are at the same place.
`Horizontal(line)`	The line's two ends are at the same height.
`HorizontalPoints(p, q)`	`p` and `q` are at the same height.
`Vertical(line)`	The line's two ends are directly above each other.
`VerticalPoints(p, q)`	`p` is directly above or below `q`.
`Fixed { point, x, y }`	The point stays at (x, y).
`Distance { a, b, value }`	Points `a` and `b` are `value` apart. Value > 0.
`Radius { geom, value }`	The circle or arc has this radius. Value > 0.
`Diameter { geom, value }`	The circle or arc has this diameter. Value > 0.

A point `p` is written `PointRef::Point(id)` for a point entity, `Start(id)` or `End(id)` for a line's ends, or `Center(id)` for a circle's or arc's center.

9.5 Solver settings and report

Setting	Meaning	Default
`tolerance`	Done when the remaining error is at most this.	`1e-10`
`max_iterations`	Give up after this many attempts.	`100`
`initial_damping`	How cautious the first steps are.	`1e-6`

Stop reason	Meaning
`Converged`	Every rule holds, within the tolerance. `converged` is `true`.
`MaxIterations`	The attempts ran out first.
`Stagnated`	No move makes it better: usually rules that conflict.
`NonFinite`	A value became infinite or not a number.

The solver uses the Levenberg–Marquardt method, a standard way to solve many equations at once; see J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., chapter 10.

9.6 File formats and units

Format	Read / write	Notes
STEP (`.step` , `.stp`), ISO 10303-21	Both	Solids only. Written in millimeters.
PNG (`.png`)	Write	Pictures from `swx-qa render`.
Saulitwerx part (`.sltwx`)	Not yet	Planned: the part with its full history.

Quantity	Unit
Length	millimeter (mm)
Area, volume	mm^2, mm^3
Angle	degrees for feature parameters such as a revolve's sweep; radians for arc angles in sketches

9.7 Keyboard and mouse

There are no keyboard or mouse controls yet, because there is no app window. They arrive with the desktop app and will be listed here.

PART V

Help

What to do when something goes wrong: symptoms, every error message you can see, common questions and known limitations.

CHAPTER 10

Help

10.1 Troubleshooting

Symptom	What to do
`cargo: command not found`	Rust isn't installed, or this terminal started before it was. Open a new terminal; if that doesn't help, repeat the Rust step in Chapter 2.
`could not find Cargo.toml`	The terminal isn't in the `saulitwerx` folder. Run `cd saulitwerx` and try again.
A build with `--features occt` fails with errors about `.hxx` files or `TK…` libraries	OpenCascade isn't installed. Install the `libocct-…-dev` packages from Chapter 2.
`swx-qa: cannot read … STEP input needs OCCT`	`swx-qa` was built without OpenCascade. Rebuild it: `cargo build --release -p swx-qa --features occt` .
`target/release/swx-qa: No such file or directory`	The tool isn't built yet. Run the build step in Chapter 2.
Rows of `*` and colored text appear when saving or reading STEP	Normal. The geometry engine prints its own report. Saulitwerx's own lines come before and after it.
The picture looks small or blurry when printed	Use a larger `--size` : about 80 pixels per centimeter of printed width.
A curved part is covered in lines	Raise `--crease` , for example to 45.

10.2 Error messages

From `swx-qa`

Every message starts with `swx-qa:` . No picture is written when there is an error.

Message	Meaning and fix
`missing command`	Nothing after `swx-qa` . Add `render` , or `--help` .
`unknown command `x``	Only `render` and `help` exist.
`unknown option `x``	A misspelled option. The help text follows the message.
`` `--size` needs a value ``	An option was last on the line without its value. The same goes for the other options.
`missing input: give --fixture <name> or --step <file.step>`	Say what to draw.
`give exactly one of --fixture and --step`	Two inputs were given. Keep one.
`missing output: give -o <out.png>`	Say where to save the picture.
`unknown fixture `x`; available: cube, cylinder, l-bracket`	Use one of the listed names.
`invalid --size `x`: expected <W>x<H>, e.g. 800x600`	Write the size as two whole numbers joined by `x` , with no spaces.
`invalid --size `x`: width and height must be > 0`	Neither side may be 0.

Message	Meaning and fix
`image size WxH must be between 1x1 and 8192x8192`	Use a smaller size.
`invalid --crease `x`: expected degrees`	Give a number, such as `45`.
`crease angle x must be between 0 and 180 degrees`	Use a value from 0 to 180.
`cannot read f: invalid input: no such file: f`	The STEP file isn't there. Check the name and the folder.
`cannot read f: kernel failure: import_step from f failed`	The file isn't valid STEP. The geometry engine prints the reason just above.
`cannot write f: …`	The picture can't be saved there, for example because the folder doesn't exist.
`mesh is empty`, `mesh has no visible extent`	The STEP file holds nothing that can be drawn.

From the geometry kernel

Message	Meaning and fix
`invalid input: <name> must be finite and > 0, got <v>`	A size, radius or height is zero, negative or not a number.
`invalid input: <name> must have finite coordinates, got …`	A position contains an infinite value or not-a-number.
`invalid input: no such file: f`	The STEP file to read doesn't exist.
`kernel failure: <operation> failed`	OpenCascade couldn't complete the operation. For a cut, check that the tool solid overlaps the part (Chapter 6).

From sketches

Message	Meaning and fix
`geometry values must be finite`	A coordinate, radius or angle is infinite or not a number.
`radius must be > 0, got r`	A circle or arc needs a positive radius.
`value must be > 0, got v`	A distance, radius or diameter constraint needs a positive value.
`unknown geometry N`	The constraint names an entity that isn't in this sketch.
`geometry N has a different kind`	Wrong kind of point or entity, such as `Center` of a line (Chapter 5).

From the document model

Message	Meaning and fix
`unknown parameter `p``	Set the parameter before a feature uses it.
`unknown feature N`	No feature with that number is in the history.
`feature N references feature M which is not earlier in the history`	A feature can use only features made before it.

10.3 Questions

Is there a version with windows and buttons? Not yet. The desktop app is a later milestone; see **Coming next**.

Can I open files from other CAD programs? Yes, if you save them as STEP first. Saulitwerx doesn't read any program's own native file format, and won't.

Can I save my design and edit it later? Not yet. Today you can save solids as STEP, which keeps the shape but not the history. Saulitwerx part files come later.

Does it cost anything? No. Saulitwerx is free and open source, under the MIT or Apache-2.0 license, at your choice.

Why millimeters? Mechanical design works mostly in millimeters, so it is the default. Choosing other units per part is planned.

10.4 Known limitations

- No desktop app, 3D view, mouse or keyboard controls yet.
- No features that turn sketches into solids yet: no extrude, revolve, fillet or hole. Solids are made from boxes, cylinders and spheres.
- No native part files; work can't be saved with its history.
- Sketches can't yet report how many degrees of freedom are left, or which constraints conflict.
- Sketch constraints are limited to coincident, horizontal, vertical, fixed, distance, radius and diameter.
- Only built on and tested on Ubuntu Linux.
- An imported STEP file becomes one shape, and can't be edited.

PART VI

Back matter

Glossary, what changed, what comes next, a one-page card, credits and licenses, and the index.

Glossary

Bounding box
The smallest box, with sides parallel to the axes, that holds a solid.

Boolean operation
Combining two solids by union, cut or intersect.

CAD
Computer-aided design: using a computer to design physical objects.

Coincident
A constraint that puts two points at the same place.

Constraint
A rule about a sketch's shape, such as "horizontal" or "these points touch".

Cut
A Boolean operation that removes one solid from another.

Degree of freedom
One way a piece of a sketch can still move. A free point has two.

Diameter
The width of a circle through its center; twice the radius.

Dimension
A constraint with a number: a distance, radius or diameter.

Dirty
Said of a feature that must be rebuilt after a change.

Edge
A curve where two faces of a solid meet.

Entity
One piece of sketch geometry: a point, line, circle or arc.

Face
One flat or curved patch of a solid's surface.

Feature
One step in building a part, such as a sketch or a cut. Features are kept in the history.

Feature tree
Another name for a part's history.

Fixed
A constraint that keeps a point at exact coordinates.

Fully constrained
A sketch whose constraints and dimensions leave nothing free to move.

Geometry kernel
The part of a CAD program that does the 3D math. Saulitwerx uses OpenCascade.

History
A part's features, in the order they were made.

Intersect
A Boolean operation that keeps only what two solids share.

Isometric
A view from a corner, looking down at an angle, so that three sides of a box show equally.

OpenCascade (OCCT)
The open-source geometry kernel Saulitwerx uses.

Parameter
A named number that features and dimensions refer to.

Parametric
Built from steps and numbers that can be changed later, with the design rebuilding to match.

Pre-alpha
Early, unfinished software: the parts work, but it isn't ready for real use.

Sketch
A 2D drawing on a plane, made of points, lines, circles and arcs.

Solid
A closed 3D shape with an inside and an outside.

Solver
The calculation that moves sketch geometry until every constraint holds.

STEP
An international standard file format (ISO 10303) for exchanging 3D parts.

Surface area
The total area of a solid's faces.

Union
A Boolean operation that joins two solids into one.

Unit
What a number measures in, such as millimeters.

Volume
How much space a solid fills.

What’s new

2026-10-22: first edition

The first edition of this manual. It covers everything on `main` today:

- Installing from source, and a first tutorial that builds, measures, saves and draws a plate with a hole.
- The ideas of parametric CAD: solids, units, sketches, constraints, dimensions, parameters, features and history.
- Sketching with the coincident, horizontal, vertical, fixed, distance, radius and diameter constraints, and the solver.
- Making and combining solids, and saving and reading STEP files.
- Drawing parts with `swx-qa render`.
- The feature history, and finding what a change rebuilds.
- Reference tables, every error message, troubleshooting, and known limitations.

Coming next

These are the next items on the public roadmap. None of them works yet. When one arrives, it moves into the chapters of this manual and into **What's new**.

Soon	What you'll be able to do
More constraints	Parallel, perpendicular, tangent, equal, midpoint and symmetric.
Degree-of-freedom report	See whether a sketch is under-, fully or over-constrained, and which constraints conflict.
Profiles	Turn closed loops in a sketch into regions, with holes, ready to become solids.
Pictures of sketches	Draw a solved sketch, with its points and dimension values, to a PNG.
Dimension parameters	Every dimension becomes a named parameter, `d0`, `d1`, …, with expressions and units.
Part features	Extrude, revolve, fillet, chamfer, shell and hole, driven by sketches.
Part files	Save a part with its full history, and open it again.
Desktop app	A window with a 3D view, a model browser, sketch mode and property panels.

Later milestones add assemblies, 2D drawings and exploded views. The full plan is in `docs/ROADMAP.md` in the source code.

Quick reference

BUILD

TERMINAL

```
cargo test --workspace
cargo build --release -p swx-qa \
  --features occt
```

DRAW A PART

TERMINAL

```
swx-qa render --step part.step -o part.png
swx-qa render --fixture cube -o cube.png
  --size 1600x1000   (default 800x600)
  --crease 45     (default 30)
```

SAMPLES

TERMINAL

```
cargo run -p swx-sketch \
  --example plate_sketch
cargo run -p swx-kernel --features occt \
  --example plate_with_hole -- plate.step
cargo run -p swx-core \
  --example feature_history
```

SKETCH CONSTRAINTS

Constraint	Keeps
Coincident	two points together
Horizontal	a line level
Vertical	a line upright
Fixed	a point at (x, y)
Distance	two points a length apart
Radius / Diameter	a circle's size

SOLIDS

Call	Gives
`make_box`	box from corner + sizes
`make_cylinder`	cylinder on +Z
`make_sphere`	sphere
`union` · `cut` · `intersect`	Boolean result
`export_step` · `import_step`	STEP file out / in

UNITS

Millimeters. Angles: degrees in feature parameters, radians in sketch arcs.

Credits and licenses

Saulitwerx is written by its contributors and is licensed under either the MIT license or the Apache License 2.0, at your choice. It is an independent project, not affiliated with or derived from any commercial CAD product.

Component	License	Notes
OpenCascade Technology (OCCT) 7.6	LGPL-2.1 with the OCCT exception	The geometry kernel. Used as a separate, dynamically linked library; not included in Saulitwerx's source.
`png`, `flate2`, `miniz_oxide`, `fdeflate`, `crc32fast`, `adler2`, `simd-adler32`, `bitflags`, `cfg-if`	MIT, Apache-2.0, Zlib or 0BSD	Write the PNG pictures of `swx-qa`.
`cc`, `shlex`, `find-msvc-tools`	MIT or Apache-2.0	Build the bridge to OpenCascade.
Typst 0.15.1	Apache-2.0	Typesets this manual.
Inter 4.0, by The Inter Project Authors	SIL Open Font License 1.1	Headings and labels in this manual. License in `docs/manual/fonts/OFL-Inter.txt`.
Libertinus Serif, DejaVu Sans Mono	SIL Open Font License 1.1; Bitstream Vera license	Body text and code in this manual, as bundled with Typst.

Every picture of a part in this manual was drawn by Saulitwerx's own renderer, `swx-qa render`, from geometry made by Saulitwerx.

Index

B

boolean operations 13
bounding box 11

C

CAD 2
coincident 12
constraint 12
cut 13

D

degree of freedom 12
diameter 12
dimension 12
dirty 24
distance 12

E

edge 11
entities 12

F

faces 11
feature 13
feature tree 13
fixed 12
fully constrained 12

G

geometry 12
geometry kernel 4, 19

H

history 13
horizontal 12

I

intersect 13
isometric 7

L

Levenberg–Marquardt 28

O

occt feature 27
OpenCascade 4

P

parameter 13
parametric 2
PointRef 17

R

radius 12
rebuild 24

S

sketch 12
solid 8, 11
solver 12
STEP 9
surface area 11
swx-qa render 26

U

union 13
unit 11

V

vertical 12
volume 11